

번호	정답	해설
1	②	따옴표와 소수점이 없는 25는 정수형이다.
2	③	11을 4로 나눈 나머지는 3이다.
3	①	문자열의 첫 번째 문자 인덱스는 0이다.
4	④	append(40)은 리스트 끝에 정수 40을 추가한다.
5	③	원소가 하나인 튜플은 쉼표가 필요하다.
6	①	딕셔너리는 키를 사용하여 값을 조회한다.
7	④	집합에는 같은 값이 중복으로 저장되지 않는다.
8	②	빈 리스트는 거짓이다. 나머지는 비어 있지 않은 문자열 또는 리스트이다.
9	③	==는 값의 동등성을 비교한다.
10	①	75는 70 이상이므로 "합격"이 출력된다.
11	④	range(4)는 0, 1, 2, 3을 차례로 제공한다.
12	②	break는 가장 가까운 반복문을 종료한다.
13	①	함수 정의에는 def를 사용한다.
14	③	input()은 입력한 숫자도 문자열로 반환한다.
15	④	Student()를 호출하여 생성한 객체가 s1이다.
16	②	//는 내림 나눗셈, /는 일반 나눗셈이다.
17	③	인덱스 1, 3, 5의 문자는 각각 O, P, T이다.
18	①	양쪽 공백을 제거한 뒤 쉼표를 기준으로 나눈다.
19	④	a와 b가 같은 리스트를 가리킨다.
20	②	복사한 리스트의 원소를 바꾸므로 원본은 유지된다.
21	③	sort()는 원본을 정렬하고 None을 반환한다.
22	①	키가 3개가 되고 점수는 85로 바뀐다.
23	④	&는 교집합이며 공통 원소는 3이다.
24	②	점수 조건과 출석 조건이 모두 참이다.
25	③	첫 조건이 참이므로 이후 elif는 검사하지 않는다.
26	①	2, 4, 6, 8을 더하면 20이다.
27	④	3을 제외한 1, 2, 4, 5를 더하면 12이다.
28	②	4를 더하기 전에 종료하므로 합은 1+2+3=6이다.
29	③	합계는 6이며, count가 4가 되면 반복이 끝난다.
30	①	짝수 2와 4에 각각 2를 곱한다.

번호	정답	해설
31	④	정의에서 받는 변수가 매개변수, 호출에서 전달하는 값이 인수이다.
32	②	함수가 5를 출력한 뒤 None을 반환하므로 두 줄이 출력된다.
33	③	첫 호출은 기본값 2, 두 번째 호출은 전달한 값 4를 사용한다.
34	①	키워드로 연결되므로 a=10, b=3이다.
35	④	반환된 튜플 (7, 10)을 x, y에 나누어 저장한다.
36	②	함수 내부의 지역 변수는 바깥의 value를 변경하지 않는다.
37	③	*args는 여러 위치 인수를 튜플로 받는다.
38	①	kwargs는 딕셔너리이며 "score"의 값은 90이다.
39	④	전달받은 7에 2를 곱하여 14를 반환한다.
40	②	두 문자열을 정수로 변환한 뒤 더하므로 15이다.
41	③	"a"는 기존 내용 뒤에 추가하는 모드이다.
42	①	with문은 블록을 벗어날 때 파일을 닫는다.
43	④	인덱스 0은 스크립트 이름, 1은 red, 2는 blue이다.
44	②	s1의 이름 변경은 s2의 이름에 영향을 주지 않는다.
45	③	__init__()는 객체 생성 과정에서 초기 상태를 설정한다.
46	①	a, b는 같은 리스트이고, c는 복사한 별도 리스트이다.
47	④	80점 이상인 민지와 서연의 이름만 추출한다.
48	②	짝수는 건너뛰고 7에서 종료한다. 합계는 1+3+5=9이다.
49	③	바깥 반복마다 j=0에서 한 번 증가하고 j=1에서 내부 반복을 종료한다.
50	①	return이 반복문 안에 있어 첫 원소를 더한 직후 함수가 종료된다.
51	④	append()는 원본을 변경하지만, 이후 대입은 지역 이름 items가 가리키는 대상만 바꾼다.
52	②	전역 count는 7이 되고, 명시적 반환값이 없어 result는 None이다.
53	③	(10+2+3)×2=30이다.
54	①	마지막 "w"모드가 이전 내용을 지우고 C를 기록한다.
55	④	70점 이상인 80과 90을 정수로 변환하여 더한다.
56	②	a는 3+2=5, b는 5를 각각 저장한다.
57	③	a는 자체 속성 "B학교"를, b는 클래스 속성 "C학교"를 사용한다.
58	①	부모의 초기화 메서드로 95를 저장하고 자식의 result()를 실행한다.
59	④	클래스에서 메서드를 호출하면서 box를 self에 직접 전달한다.
60	②	점수가 70, 85, 73으로 바뀌어 세 객체 모두 합격한다.

61. 리스트와 튜플 비교

- 공통점: 원소의 순서가 있으며 인덱싱과 슬라이싱이 가능하다.
- 차이점: 리스트는 원소를 추가·삭제하거나 다른 값으로 대입할 수 있지만, 튜플은 원소 대입·삭제가 불가능하다.

```
numbers_list = [10]
numbers_tuple = (10,)
```

62. 변수의 참조와 복사

출력 결과:

[99, 20]

[10, 20, 30]

b = a는 리스트를 복사하지 않으므로 a와 b가 같은 리스트를 가리킨다. 따라서 b[0]을 변경하면 a에서도 변경된 값이 보인다.

c = a.copy()는 별도 리스트를 만든다. 이 코드에서는 c에 원소를 추가해도 a의 구조가 바뀌지 않는다.

63. 조건문 작성

```
if score >= 90:
    print("A")
elif score >= 80:
    print("B")
elif score >= 70:
    print("C")
else:
    print("D")
```

높은 점수부터 검사한다. 앞 조건을 통과하지 않았으므로 score >= 80분기에 들어온 점수는 자동으로 90점 미만이다.

64. 반복문과 조건문 작성

```
scores = [55, 80, 70, 95, 60]
total = 0
count = 0
for score in scores:
    if score >= 70:
        total += score
        count += 1
print(total, count)
```

출력 결과:

245 3

65. print()와 return 비교

출력 결과:

8

None

8

show(4)는 8을 화면에 출력하지만 그 값을 반환하지 않아 x에는 None이 저장된다.

calculate(4)는 8을 반환하므로 y에는 8이 저장된다. 함수 호출 자체는 화면에 출력하지 않으며, 마지막 print(y)에서 출력한다.

66. 기본 매개변수를 사용하는 함수 작성

```
def check_pass(scores, standard=70):  
    average = sum(scores) / len(scores)  
    if average >= standard:  
        return "합격"  
    return "불합격"
```

호출 결과:

```
print(check_pass([80, 70, 90])) # 합격  
print(check_pass([80, 70, 90], 85)) # 불합격
```

평균은 80점이다. 첫 호출은 기본 기준인 70점, 두 번째 호출은 전달한 기준인 85점과 비교한다.

67. 파일 쓰기와 읽기

```
with open("review.txt", "w", encoding="utf-8") as file:  
    file.write("파이썬 복습 완료")  
with open("review.txt", "r", encoding="utf-8") as file:  
    text = file.read()  
print(text)
```

- "w": 파일이 있으면 기존 내용을 지우고 새로 쓴다.
- "a": 기존 내용을 유지하고 끝에 추가한다.
- 두 모드 모두 파일이 없으면 새로 만든다.

68. 프로그램 실행 인수 분석

출력 결과:

20

항목	값
sys.argv[0]	"add.py"
sys.argv[1]	"12"
sys.argv[2]	"8"

실행 인수는 문자열로 전달된다. 숫자 덧셈을 하려면 int()로 정수로 변환해야 한다. 변환 없이 두 문자열을 더하면 "128"이 된다.

69. 클래스 작성

```
class Student:  
    def __init__(self, name, score):  
        self.name = name  
        self.score = score  
    def result(self):  
        if self.score >= 70:  
            return "합격"  
        return "불합격"  
student = Student("민지", 85)  
print(student.name, student.result())
```

출력 결과:

민지 합격

self는 해당 인스턴스 메서드를 호출한 객체를 받는 매개변수이다. 위 코드에서 student.result()를 호출하면 self에는 student객체가 전달된다.

70. 상속과 오버라이딩 분석

출력 결과:

서연 장학 대상

1. 부모 클래스:Student
자식 클래스:ScholarshipStudent
2. 자식 클래스가 자체 `__init__()`를 정의하지 않았으므로 상속받은 부모의 `__init__()`를 사용하여 이름과 점수를 저장한다.
3. 자식 클래스에서 부모와 같은 이름인 `result()`메서드를 다시 정의한 부분이 오버라이딩이다. `s.result()`를 호출하면 자식 클래스의 메서드가 실행된다.